計算機視覺期末專題

DIY 數字油畫

系級 : 資工三

S1254001 鄭友友

S1254019 賴昱翰

S1254029 黃幼妤

S1254042 林芷丞

目錄

簡介:

(一) 設計理念

(二) 主題介紹

程式碼:

(三) 函式庫

實作與說明:

(四) 版本一

(五) 版本二

困難與解決方法:

(六) UI

(七) 版本一

(八) 版本二

實作成果:

(九) 版本圖片比較

(十) 結論

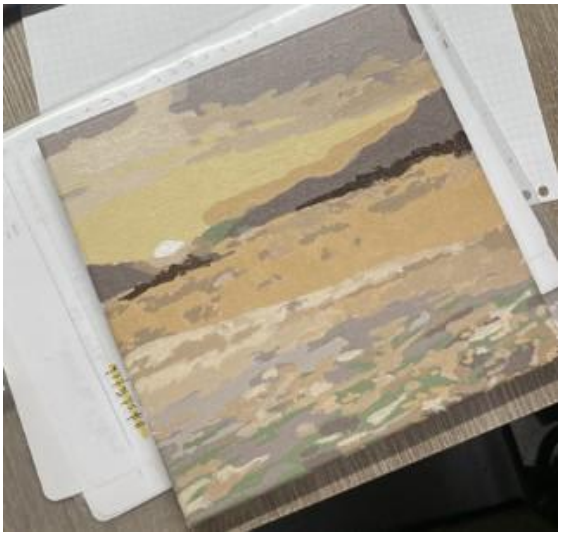
簡介：

（一）設計理念

會選擇數字油畫作為專題主要有兩個動機。第一是市面上的數字油畫價格普遍不低，長期購買下來會是一筆不小的花費。第二個原因則是存在「踩雷」的風險。以組員的過往經驗為例，曾經在購買數字油畫時，被示意圖中夢幻的配色所吸引，但實際繪製過程中卻發現成品與原圖差距越來越大，最後完成後才發現它所提供的顏料中，有超過一半的顏色與示意圖不符，導致最終作品完全失去原本的美感，既花費時間又浪費金錢。

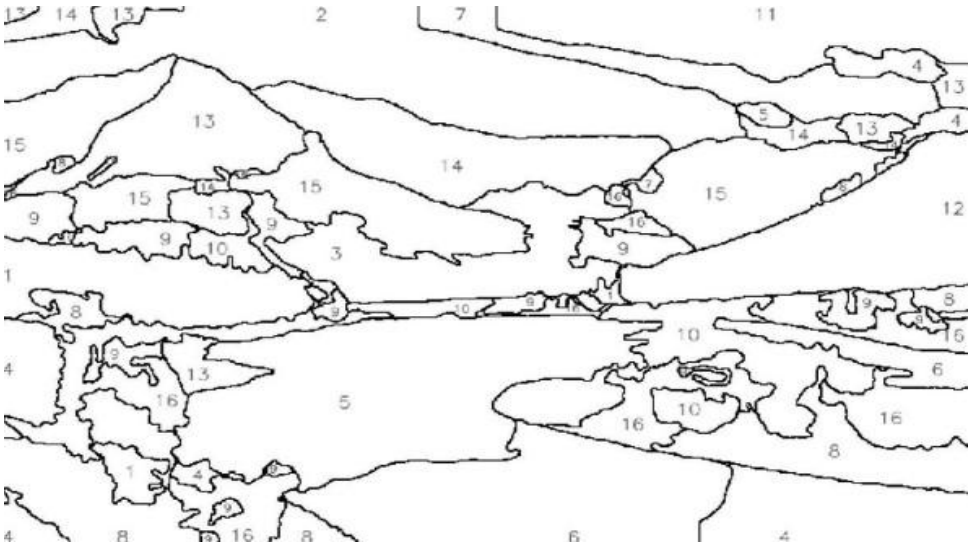
因此我們就在思考，若我們能在實際繪畫之前，提供一個程式讓使用者能先上傳圖片，即時產生處理後的數字油畫圖，並在線上進行填色模擬，預先看到完成後的效果，甚至能自行選擇不同的數字油畫風格，便能有效避免購買後才後悔的情況。此外，使用者也可以將產生的數字圖印出來，自行搭配喜歡的顏料進行創作，不僅自由度更高，長期畫下來也能更省錢。





（二）主題介紹

- 畫布結構：畫布上佈滿許多封閉色塊。
- 編號指示：每個色塊內部都對應特定的顏色編號。
- 創作過程：繪畫者只需要在畫布區塊塗上對應數字的顏色即可。



程式碼 ：

（三）函式庫

1. Python: 3.13.3
2. opencv-contrib-python: 4.12.0.88
3. numpy: 2.2.6
4. pillow: 11.2.1
5. scipy: 1.16.2

由於版本一和版本二的程式碼篇幅太長，整個貼上來會至少30頁，因此我們上傳於雲端，就不顯示於本文件檔。

實作與說明 ：

（四）步驟 - 版本一

1. 前處理 **：**

a. 變圖片比例

b. Mean Shift 濾波: 每個像素視為一個包含 位置（x, y）與 顏色（R, G, B）的 5維資料點。

- 定義數據空間和搜尋視窗（sp:空間半徑、sr:色彩半徑）
- 計算區域內加權平均值(距離、顏色相似度）
- 更新重心位置
- 迭代直到收斂（重心位子不變）
- 歸類色塊

2. 影像處理 ：

K-means: 歸納出 16 種代表色，決定畫面的基本色調與分區。

3. 後處理 ：

a. 型態學開運算 (Morphological Opening): 移除K-Means產生的單像素雜訊。

b. 區域合併: 消除過小色塊，確保不會出現塗不到的小格子。

c. 中值模糊 (Median Blur): 把色塊邊界的鋸齒磨圓，讓色塊邊界看起來更自然。

d. 計算顏色歸屬 (Distance Matching):視覺上的顏色（RGB）轉換成色卡編號（ID）

$$M(x) = \frac{\sum K(P_i - P) \cdot P_i}{\sum K(P_i - P)}$$

e. 使用自訂義函數再次移除過小區域。

4. 輸出 ：

a. Sobel 邊緣偵測 : 在兩個不同顏色的色塊之間，精準的畫出分界線。

b. 標記數字 : 使用Distance Transform找出中心位置。

（五）步驟 - 版本二

1. 前處理 :

   a. 改變圖片比例。

   b. 雙邊濾波: 去噪同時保留邊緣，避免顏色混合過度。

   c. 中值模糊: 針對孤立的小斑點或雜訊做最後清理。

2. 影像處理 :

   a. SLIC: 圖像被分割成大小均勻、邊界貼合輪廓的超像素區域。

   b. K-Means: 對圖像顏色進行聚類，找出 16 種代表色。

   c. 顏色映射: 將每個 SLIC 超像素的「平均顏色」與 K-Means 找出的 16 種中心色進行比對，將該超像素歸類為最接近的那一種顏色。

3. 後處理 :

   a. 區域合併: 小面積被合併為周圍占比最多的label

   b. 邊界平滑: 放大 -> 中值模糊 -> 縮小

      - 圖像放大2倍，使原本鋸齒狀的邊界線變得高解析度
      - 中值模糊 : 模糊平滑邊界線，減少不自然鋸齒
      - 縮小回原始尺寸，保留平滑效果

4. 輸出 :

   a. 邊緣偵測: 利用膨脹減去侵蝕得到區域邊界，有做標籤可明確被偵測。

   b. 標記數字: 使用Distance Transform找出中心位置。

困難與解決方法 ：

(六)UI

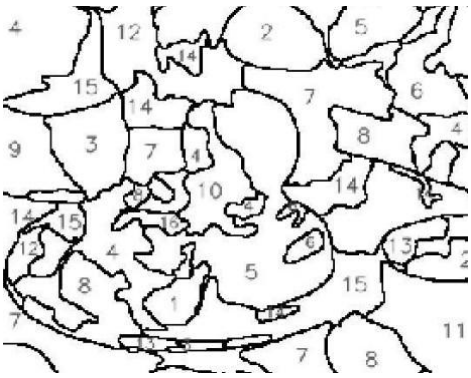
Q: 線稿常存在許多面積非常微小，難以點擊或識別的色塊。

A: 為了確保使用者能輕鬆、準確地將顏色填入這些區塊較小的地方，我們加入了動態的視圖控制功能。

- 放大縮小 ： 滾輪
- 右鍵 ： 抓取移動
- 左鍵 ： 填色

Q ： 圖片難易度適應 ： 區塊小且密 vs 大面積。

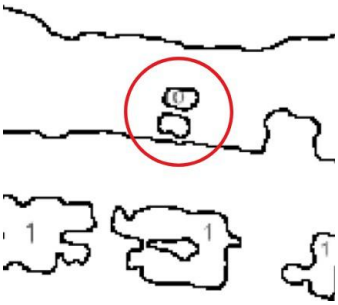
- 模式一 ： 著色單一區塊
- 模式二 ： 將相同數字顏色的區塊一起填色



(七) 版本一

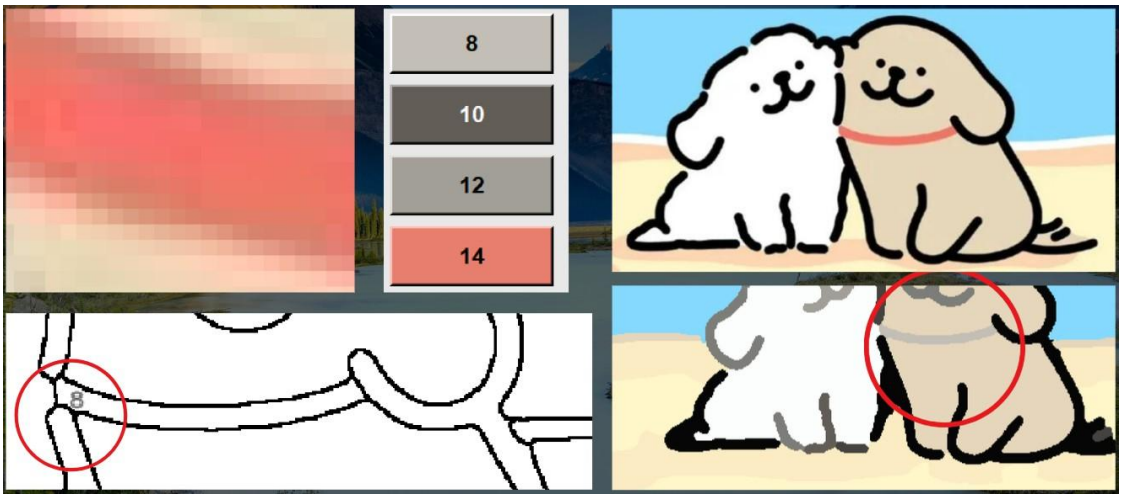
Q: 出現「幽靈區塊」有框線卻無數字、無法點擊的問題。

A: 使用自定義的後處理函數，在計算歸屬步驟後再次移除過小區域 。



Q: 小區域出現顏色判斷錯誤的問題。

A: 解決方法 ： 參數調整（ 0.04 -> 0.01 ）



（八）版本二

Q: 在merge後就直接輸出，會使圖形稜稜角角的。

A: 放大後，使用兩次中值模糊，可使菱角被去除。

Q: 在高解析度圖片上會執行過久以及切出的格子過小

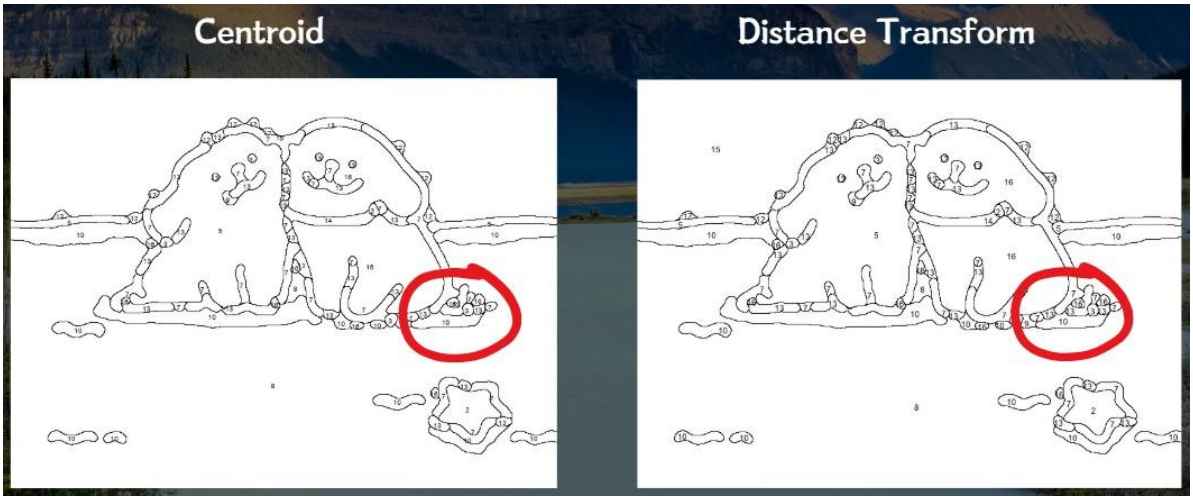
- 圖片太大將解析度等比例縮小
- 限制superpixel最多切 2000 並且region_size最小為10
- 在region合併時，只保留面積>30的





Q: 在標數字的時候，使用Centroid有時會標到區域外

A: 使用Distance Transform的方式來找出最大值為中心點。



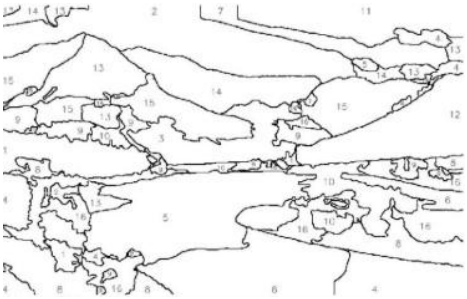
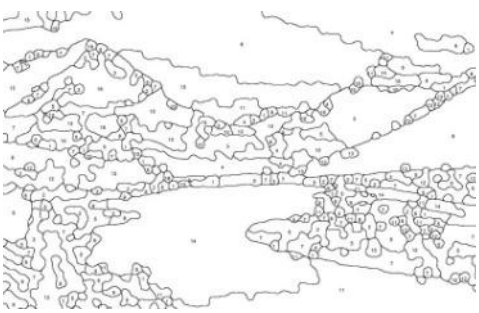
實作成果 ：

（九）版本圖片比較

1. 風景

➤ 版本一 ： 區塊切的比較大，邊界比較平滑（適合新手）

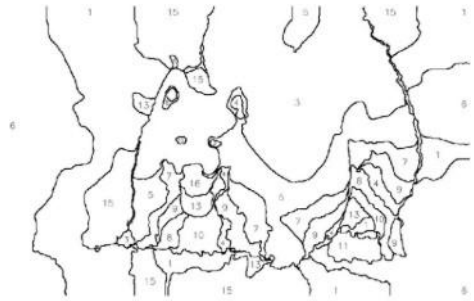
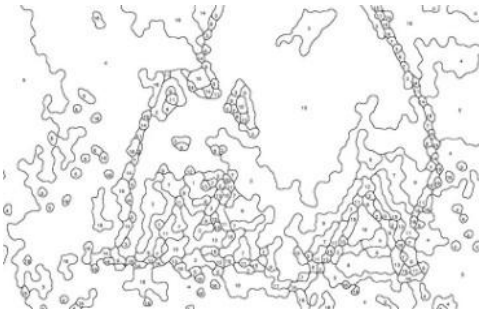
➤ 版本二 ： 區塊切的比較小，保留較多細節（難度較高）

版本一		
版本二		

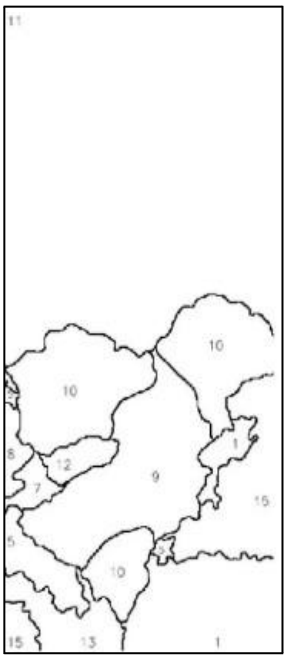
2. 大面積相似色背景

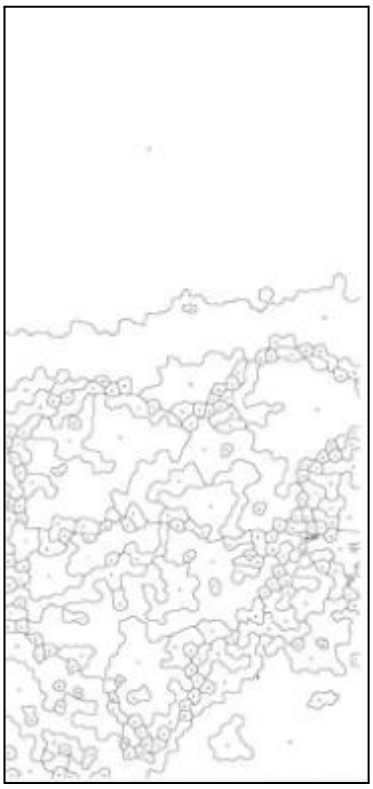
➤ 版本一 ： 容易出現細長、線條感比較明顯的區塊

➤ 版本二 ： 不論是區塊或是相對較扁的形狀，皆有保留一定的寬度，整體會很像鵝卵石的感覺

版本一		
版本二		

3. 天空

版本一		

版本二		

（十）結論

這份DIY數字油畫，將這學期課堂上的抽象理論，轉化為實際應用。

聽老師講解那些複雜的數學公式與原理時，我們對這些知識其實是一知半解的。直到開始(靠自己)動手寫程式，這些知識互相連接運作，才變得具體化。

在實作過程中，我們遇到了很多上課沒講過的突發狀況。

為了解決這些問題，我們必須理解(ex.k-mean)、補充(ex.Mean Shift 濾波)，並自己加入設計邏輯來優化它(ex.UI優化使用)。

雖然過程充滿挑戰，但當最後看到成果時，原來『原來數學、原理真的能變成藝術品』。